Министерство науки и высшего образования Российской Федерации
Федеральное государственное бюджетное образовательное учреждение высшего образования
«Московский государственный технический университет имени Н.Э. Баумана (национальный исследовательский университет)»
(МГТУ им. Н.Э. Баумана)

ФАКУЛЬТЕТ Информатики и систем управления

КАФЕДРА Теоретической информатики и компьютерных технологий

ОТЧЁТ ПО ПРОИЗВОДСТВЕННОЙ ПРАКТИКЕ

Студент: Нащекин Никита Денисович

Группа: ИУ9-42Б

Тип практики: производственная

Название предприятия: «Военная академия РВСН им. Петра Великого»

Студент _____ Нащекин Н. Д.
подпись, дата *фамилия, и.о.*

Рекомендуемая оценка: _____
Руководитель практики
от предприятия: _____ Филатов В. И.
подпись, дата *фамилия, и.о.*

Руководитель практики _____ Коновалов А. В.
подпись, дата *фамилия, и.о.*

Оценка _____

2024 г.

Оглавление

1. Введение с. 3
2. Основная часть с. 4
3. Заключение с. 14
4. Список использованных источников с. 15
5. Приложение с. 16

Введение

Целью практики является доработка и улучшение ранее созданной системы мониторинга радиоэфира на предмет наличия вещания в заданной пользователем полосе частот, а также внедрение в систему нейронной сети для эффективного определения класса вещания.

Для достижения поставленной цели необходимо выполнить ряд задач:

1. Добавление в систему лёгкой в использовании настройки конфигурации поиска вещания.
2. Сбор необходимого количества данных для качественного обучения нейронной сети.
3. Обучение нейронной сети и её тестирование на части уже собранных и новых данных.
4. Разработка клиентского приложения для подключения основной системы к нейронной сети.
5. Внесение изменений в логику работы основной системы, включающих в себя возможность выбора схемы работы системы (определение наличия вещания классическим алгоритмом или с помощью нейронной сети) и использование сжатого массива, содержащего данные об эфире в заданной полосе частот, для ускорения работы программы.
6. Проведение испытаний системы.

Основная часть

Как было сказано во введении, за основу была взята имеющаяся система мониторинга радиоэфира, разработанная в ходе летней проектно-технологической практики 2023 года.

В основе этого комплекса лежит программно-определяемая радиосистема HackRF One, которая позволяет контролировать полосу частот от 1 МГц до 6 ГГц, обладает полосой пропускания в 20 МГц, а также имеет API, позволяющий использовать её в различных средствах обработки радиоэфира [1]. Эта радиосистема отправляет в разработанную на языке C++ программу данные о текущем состоянии радиоэфира в заданной полосе частот, после чего производится быстрое преобразование Фурье с использованием библиотеки fftw3, а затем полученный массив мощностей сигнала анализируется на предмет наличия вещания [2]. Внешний вид HackRF One представлен на рисунке 1.



Рисунок 1 — внешний вид HackRF One

В исходной программе алгоритм поиска вещания был следующим: использовалось пороговое значение мощности, определяемое экспериментально, при превышении которого проводилась проверка наличия превышения мощности через 100 и 200 кГц, и в случае вновь найденного всплеска выносился вердикт о наличии вещания. Такой алгоритм оказался низкоэффективным ввиду большого размера массива (минимум 131072 значений). Кроме того, в силу особенностей программно-определяемых радиосистем, даже при наличии широкополосного вещания был шанс при повторной проверке мощности с шагом в 100 кГц попасть в значение, которое не превышает пороговое, тем самым пропустив имеющееся вещание в эфире.

Для решения этой проблемы было решено воспользоваться алгоритмом сжатия массива мощностей, который сохранит основные структурные особенности массива, при этом значительно уменьшив сложность требуемого алгоритма поиска вещания [3]. Код функции сжатия массива приведён на листинге 1.

Листинг 1 — код функции сжатия массива.

```
vector<double> PruneArray(vector<double> data, int newSize) {
    if (newSize >= data.size()) {
        return data;
    }

    double factor = static_cast<double>(data.size()) /
                                        static_cast<double>(newSize);
    vector<double> pruned(newSize);

    for (int i = 0; i < newSize; ++i) {
        int start = static_cast<int>(floor(static_cast<double>(i) * factor));
        int end = static_cast<int>(min(ceil(static_cast<double>(i + 1) *
                            factor), static_cast<double>(data.size())));

        int maxIndex = start;
        double maxValue = data[start];
        for (int j = start + 1; j < end; ++j) {
            if (data[j] > maxValue) {
                maxValue = data[j];
                maxIndex = j;
            }
        }
        pruned[i] = data[maxIndex];
    }

    return pruned;
}
```

В ходе проведения испытаний работы алгоритма сжатия был выбран новый размер массива в 1000 значений. Пример работы алгоритма показан на рисунках 2 и 3. Здесь с помощью gnuplot были выведены на график зависимости мощности от частоты исходный и сжатый массивы.

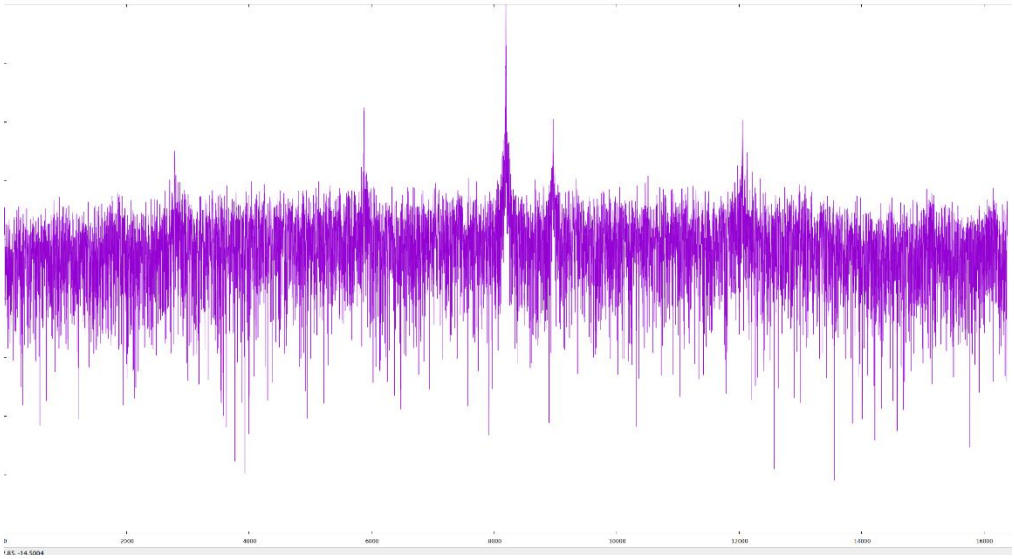


Рисунок 2 — графическое представление исходного массива

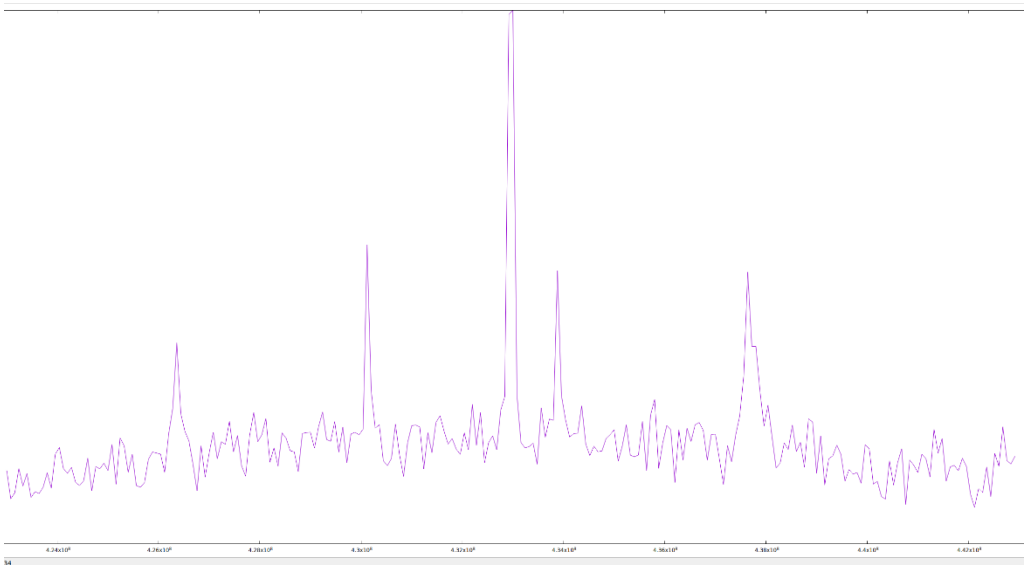


Рисунок 3 — массив после применения алгоритма сжатия

Как видно, сжатие сохранило вид спектра сигнала, а также избавило от необходимости обработки статичных узких пиков, из-за которых и нужна была проверка превышения порогового значения через 100 и 200 кГц, поскольку эти пики “растворились” после применения алгоритма сжатия.

Благодаря использованию алгоритма сжатия время обработки одного массива уменьшилось с 1 секунды в среднем до 30 миллисекунд в среднем.

После решения проблемы с алгоритмом поиска передачи в радиоэфире был реализован удобный пользователю выбор полос частот, которые должна обрабатывать программа. В конфигурационный файл config.txt записываются диапазоны контролируемых частот в МГц. В каждой строке файла может быть указана либо одиночная частота, например, “430”, и в таком случае программа будет контролировать полосу шириной 20 МГц с центром в указанной частоте, либо диапазон частот через дефис, например, “2400-2500”, в таком случае программа автоматически прибавляет 10 МГц к левой границе, вычитает 10 МГц из правой границы и с шагом в 20 МГц, начиная от обновлённой левой границы, записывает в вектор частот на сканирование нужные диапазоны, пока не будет достигнута обновленная правая граница. В вырожденном случае, когда ширина указанного диапазона меньше 20 МГц, выбирается среднее значение между границами. В самом файле конфигурации для удобства добавлен комментарий с инструкцией по его использованию. Пример файла конфигурации показан на рисунке 4.

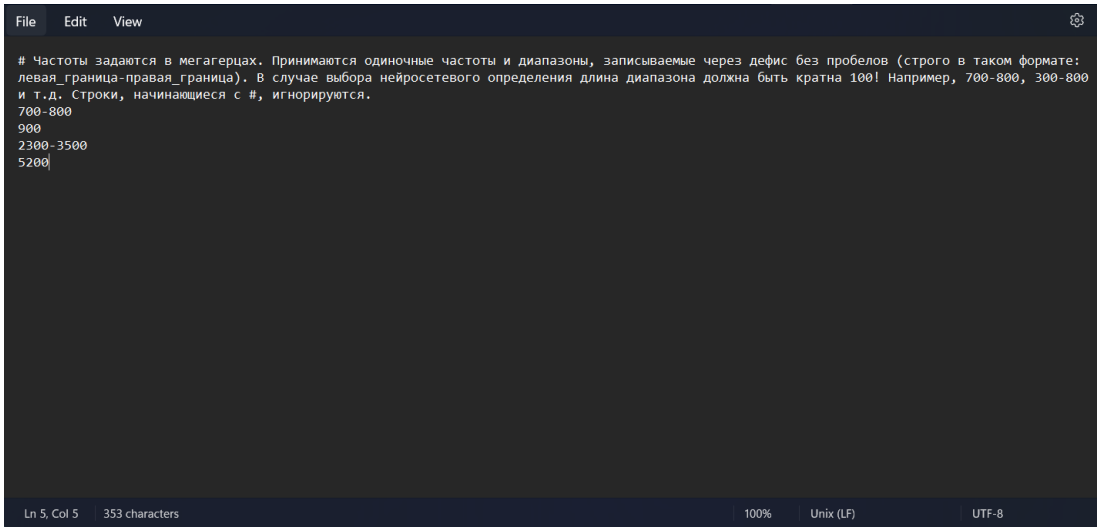


Рисунок 4 — пример файла конфигурации

Следующим этапом работы стал сбор данных для обучения нейронной сети, предназначенной для определения класса сигнала в радиоэфире. Исходные коды нейронной сети были предоставлены выпускницей кафедры

ИУ9, которая разработала её в рамках выпускной квалификационной работы. Были использованы 4 различных метода: метод k-ближайших соседей, метод логистической регрессии, линейный дискриминантный анализ и дерево решений [4]. Сама система реализована в виде локальной сетевой службы. Сервер, который управляет нейронной сетью, при запуске ожидает подключение клиента. При включении клиента необходимо выбрать текстовый файл со строкой, содержащей данные о текущем состоянии радиоэфира, которая будет отправлена на сервер. После обработки строки сервер возвращает клиенту номер класса, которому с наибольшей вероятностью соответствует переданный сигнал. На рисунке 5 представлен пример работы клиентской части нейронной сети.

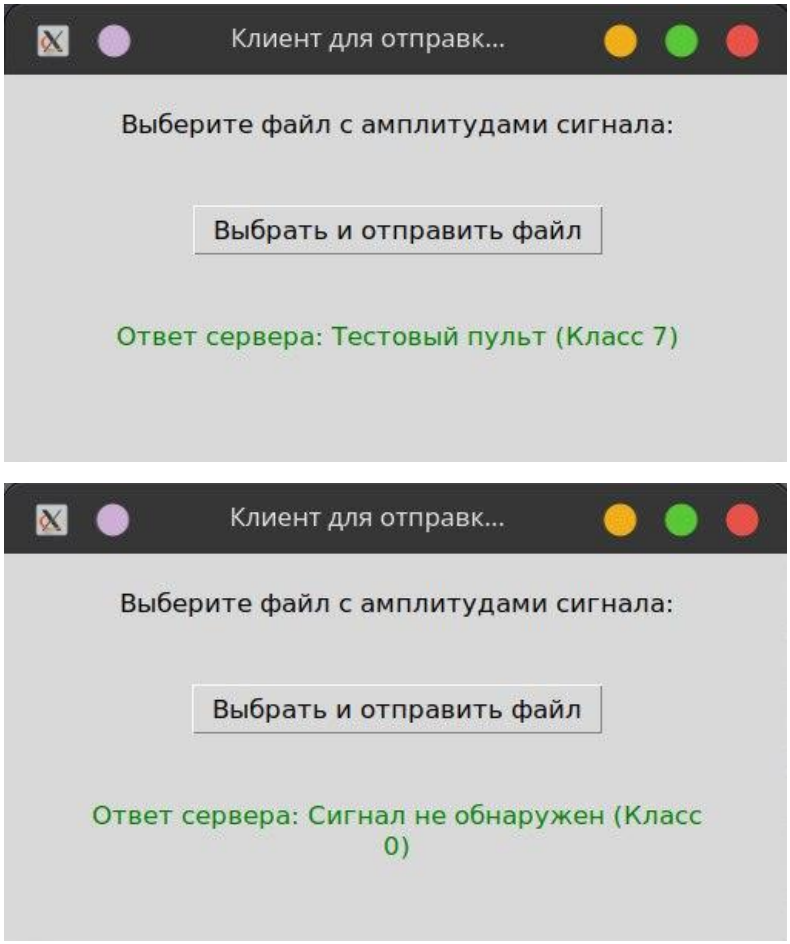


Рисунок 5 — пример работы клиента к нейронной сети

Ещё до прохождения этой практики мне была поставлена задача разработки программного обеспечения для записи нужных сигналов для обучения вышеуказанной нейронной сети. Несколько классов сигналов,

которые должна была распознать эта нейросеть, имели ширину вещания более 20 МГц, вплоть до 100 МГц, поэтому было принято решение записывать сигнал частями по 20 МГц (как было сказано выше, максимальная ширина полосы, которую может принять HackRF One, — 20 МГц) и собирать полосу в 100 МГц из пяти частей. Готовая полоса в виде массива мощностей сигнала записывалась в текстовый файл. Для хорошего обучения требовалось записать несколько тысяч строк для каждого класса сигнала [5]. Другой студент записывал пять классов сигналов с использованием моей программы и передавал собранные данные выпускнице для обучения на них нейронной сети.

Была выполнена доработка исходной программы обучения. Изначально данные о состоянии эфира при наличии сигнала каждой категории и при его отсутствии были записаны в один и тот же класс, поэтому нейронная сеть в режиме работы с реальными данными не могла корректно определить наличие сигнала какого-либо из классов.

Для решения этой проблемы я переделал программу обучения. В данные для каждого из классов сигнала были записаны только массивы, описывающие состояние эфира, в котором происходило вещание сигнала соответствующего класса. Все остальные массивы без вещания были собраны в отдельную категорию, которая описывает отсутствие сигнала какого-либо из пяти классов. Этот дополнительный класс было решено расширить добавлением большего количества данных. Для этого на частотах 700–800 МГц, 430 МГц, 2.4–2.5 ГГц и 5.3–5.4 ГГц было записано достаточное количество данных о состоянии радиоэфира при отсутствии в эфире сигналов рассматриваемых классов. Обновлённый код функции обучения модели представлен в листинге 2 в разделе “приложение” [6].

После переобучения нейронной сети и её тестирования на записанных данных, которые не использовались в обучении, оказалось, что лучше всего себя показывает метод трёх ближайших соседей. В большинстве случаев эта модель правильно определила класс сигнала, в то время как результаты работы других моделей сравнимы со случайным выбором класса. Теперь сервер

использует только одну модель для определения категории сигнала. Итоговый код сервера представлен на листинге 3 в разделе “приложение” [7].

Для удобного тестирования работы нейронной сети был создан дополнительный седьмой класс, который описывает сигнал с небольшого радиопульта, работающего на частоте 433 МГц. С помощью программы записи эфира была собрана информация о вещании с пульта и добавлена к данным для обучения нейронной сети распознаванию седьмого класса сигналов.

Для интеграции нейронной сети в нашу систему была изменена логика работы клиентской и серверной части. Теперь сервер, управляющий нейронной сетью, принимает не текстовый файл, а строку, содержащую мощности сигналов, которую через сокет передаёт единая система. После обработки строки сервер отправляет системе цифру с номером класса сигнала. Также сервер не разрывает соединение с клиентом после обработки строки, как это было ранее, а продолжает принимать и обрабатывать следующие строки.

Проблемой в использовании нейронной сети стало то, что её обучение проводилось на массивах, содержащих информацию о полосе радиоэфира шириной 100 МГц, а HackRF One может сканировать только полосу в 20 МГц. Было принято решение сканировать эфир 5 раз со смещением на 20 МГц и собирать один массив из пяти, после чего отправлять этот собранный массив на сервер.

Предыдущая логика работы системы не подразумевала “склеивание” массивов, сканирование проводилось на полосах шириной 20 МГц. Для сравнения эффективности классического алгоритма обнаружения нужных классов сигналов и нейросетевой модели была реализована возможность выбора режима работы программы. При использовании классического алгоритма сервер, управляющий нейронной сетью, запускать не требуется.
Также для классического метода были более точно определены пороги срабатывания системы для каждой из нужных полос частот. Испытания обоих методов должны пройти в ближайшее время.

При тестировании нейросетевого режима оказалось, что иногда строка передаётся с ошибкой или сервером принимается сразу две строки, которые склеиваются в одну. Поскольку обучение модели проводилось на строках фиксированной длины, была добавлена обработка ошибок и проверка длины принятой строки. Строки, не соответствующие заданному формату, игнорируются.

Поскольку нейронная сеть обучена на полосах в 100 МГц, была изменена логика обработки файла конфигурации. Теперь, если выбран нейросетевой режим работы, все диапазоны частот, задаваемые в конфигурационном файле, должны быть кратны 100 МГц. Если же задана единичная частота, то берётся отступ в 50 МГц влево от неё и сканируется полоса шириной 100 МГц с началом в отступе. После обработки файла конфигурации заполняется вектор сканируемых частот, для каждой полосы в 100 МГц в вектор поступает 5 центральных частот составных полос в 20 МГц.

При тестировании работы системы с радиопультом выяснилось, что, когда сигнал пульта находится в том же месте полосы, в котором он был при записи датасета, в 70–80% случаев класс сигнала определяется корректно. Но если в файле конфигурации полоса частот выбрана так, что сигнал пульта находится в другом месте полосы, седьмой класс сигналов, соответствующий тестовому радиопульту, практически не определяется. В связи с этим был пересобран набор данных для обучения нахождению седьмого класса. Для этого была изменена программа записи сигналов. Теперь в ней при каждом сканировании начальная частота сдвигается на 1 МГц. После 80 таких шагов, то есть после смещения на 80 МГц, сдвиг начинается в обратную сторону. С помощью такого решения записываемый сигнал появляется в разных частях полосы, благодаря чему нейронная сеть сможет научиться распознавать заданный класс сигнала в любой ситуации. Обновленной программой записи сигналов были записаны 1000 новых строк, которые затем были добавлены в датасет.

После переобучения и нового тестирования сигнал с радиопульта стал обнаруживаться в 70% случаев независимо от его расположения в исследуемой полосе частот. На рисунке 6 представлен результат работы нейросетевого режима классификации сигналов.

```
Частота: 430 МГц; Сообщение от сервера: Сигнал не обнаружен (Класс 0)
Частота: 430 МГц; Сообщение от сервера: Сигнал не обнаружен (Класс 0)
Частота: 430 МГц; Сообщение от сервера: Тестовый пульт (Класс 7)
Частота: 430 МГц; Сообщение от сервера: Тестовый пульт (Класс 7)
Частота: 430 МГц; Сообщение от сервера: Тестовый пульт (Класс 7)
Частота: 430 МГц; Сообщение от сервера: Тестовый пульт (Класс 7)
Частота: 430 МГц; Сообщение от сервера: Сигнал не обнаружен (Класс 0)
Частота: 430 МГц; Сообщение от сервера: Сигнал не обнаружен (Класс 0)
Частота: 430 МГц; Сообщение от сервера: Сигнал не обнаружен (Класс 0)
```

```
Частота: 430 МГц; Сообщение от сервера: Тестовый пульт (Класс 7)
Частота: 430 МГц; Сообщение от сервера: Тестовый пульт (Класс 7)
Частота: 430 МГц; Сообщение от сервера: Сигнал не обнаружен (Класс 0)
Частота: 430 МГц; Сообщение от сервера: Тестовый пульт (Класс 7)
Частота: 430 МГц; Сообщение от сервера: Сигнал не обнаружен (Класс 0)
Частота: 430 МГц; Сообщение от сервера: Тестовый пульт (Класс 7)
Частота: 430 МГц; Сообщение от сервера: Сигнал не обнаружен (Класс 0)
```

Рисунок 6 — результат работы системы

На верхнем рисунке при транслировании сигнала с радиопульта каждое сканирование выдало верный результат. На нижнем рисунке видно несколько ошибок, но в целом результат работы приемлемый.

При выполнении практики мне выдавались дополнительные задачи. В частности, для практики нынешних первокурсников мне нужно было добавить в нашу таблицу Devices с устройствами в базе данных на сервере, разработанной в ходе прошлой летней практики, были добавлены новые поля longitude и latitude (широта и долгота). Для каждого устройства были добавлены нужные координаты. Также было добавлено несколько новых устройств в вышеуказанную таблицу и созданы новые таблицы для каждого из устройств. Впоследствии поступило задание изменить координаты некоторых устройств на новые.

Также мне была выдана задача разработки программного обеспечения, которое должно принимать строку с информацией через serial порт, отправляемой, например, с микроконтроллера Arduino или Node MCU. После приема строки программа должна найти мощность сигнала на некоторой заданной частоте, добавить это значение к строке и вывести итоговую строку в лог. Итоговая программа запрашивает нужную частоту, выводит доступные serial порты и ожидает на вход порт, с которого должна приходить информация. Итоговые строки выводятся в текстовый файл.

Заключение

В ходе летней производственной практики была проделана большая работа: пришлось разобраться со сбором датасетов для обучения нейронных сетей и, собственно, с самим обучением и тестированием нейронных сетей. Пришлось несколько раз пересобирать датасеты для некоторых классов сигналов для более эффективной классификации, а также разрабатывать новую логику функционирования системы.

Параллельно работе над основным заданием выполнялись небольшие подзадачи, нужные для продвижения работы нашей студенческой команды над проектом: изменение конфигурации базы данных, заполнение в ней новых требуемых таблицы, создание небольших программ, таких как программа приема данных из serial порта, а также небольшая переработка установщика основной системы.

В скором времени должны пройти испытания нашего комплекса, в ходе которых будут выявлены возможные недостатки для дальнейшего их устранения и улучшения работы системы.

В ходе выполнения поставленных задач я впервые столкнулся с работой с нейронными сетями и их обучением. Это очень перспективное направление, и я рад, что уже на начальном уровне разобрался с созданием нейросетей.

Список использованных источников

1. Способ перехвата GSM-сигналов с помощью SDR-платформы HackRF ONE
Николаев В.В., Михайлов В.Э. Телекоммуникации и информационные технологии. 2021. Т. 8. № 1. С. 5-12.
2. Преобразование Фурье и быстрое преобразование Фурье
Кошелева Д.Д., Доронина А.В. Инновации. Наука. Образование. 2021. №38. С. 626-632.
3. Структуризация и сжатие больших информационных массивов для ускорения процессов обработки.
Кошелев В.Н.
НИР: грант № 93-01-00467. Российский фонд фундаментальных исследований. 1993.
4. Машинное обучение.
Горбаченко В.И., Савенков К.Е., Малахов М.А. Учебное пособие / Москва, 2023.
5. Сбор и обработка данных для обучения нейронной сети.
Перфильев В.А.
В книге: Значение научных студенческих кружков в инновационном развитии агропромышленного комплекса региона. Сборник научных тезисов студентов. п. Молодежный, 2023. С. 356-357.
6. Машинное обучение с применением Python и Scikit-learn.
Ситдиков Д.Р., Ильина Л.А.
В сборнике: Информатика и вычислительная техника. Сборник научных

трудов. Посвящается 50-летию Чувашского государственного университета имени И.Н. Ульянова . Чебоксары, 2017. С. 167-170.

7. https://habr.com/ru/articles/666534/ (дата обращения: 14.07.2024)

Приложение

Листинг 2 — код новой функции обучения моделей

```
def train():
    # загрузка данных
    H1, X1_0 = read_data(a=2290, b=2390, filename='data/class_1\
/TX=OFF/low_spetrum.txt')
    H2, X2_0 = read_data(a=2210, b=2310, filename='data/class_2\
/TX=OFF/low_spetrum.txt')
    H3, X3_0 = read_data(a=850, b=950, filename='data/class_3\
/TX=OFF/low_spetrum.txt')
    H4, X4_0 = read_data(a=700, b=800, filename='data/class_4\
/TX=OFF/low_spetrum.txt')
    H5, X5_0 = read_data(a=2630, b=2730, filename='data/class_5\
/TX=OFF/low_spetrum.txt')
    H6, X6_0 = read_data(a=2400, b=2500, filename='data/class_6\
/TX=OFF/low_spetrum.txt')
    H7, X7_0 = read_data(a=430, b=530, filename='data/no_signal.txt')

    H1, X1_1 = read_data(a=2290, b=2390, filename='data/class_1\
/TX=ON/low_spetrum.txt')
    H2, X2_1 = read_data(a=2210, b=2310, filename='data/class_2\
/TX=ON/low_spetrum.txt')
    H3, X3_1 = read_data(a=850, b=950, filename='data/class_3\
/TX=ON/low_spetrum.txt')
    H4, X4_1 = read_data(a=700, b=800, filename='data/class_4\
/TX=ON/low_spetrum.txt')
    H5, X5_1 = read_data(a=2630, b=2730, filename='data/class_5\
/TX=ON/low_spetrum.txt')
    H6, X6_1 = read_data(a=2400, b=2500, filename='data/class_6\
/TX=ON/low_spetrum.txt')
    H7, X7_1 = read_data(a=430, b=530, filename='data/pult_430.txt')

    steps = [200, 100, 50, 25, 15, 10, 5]
    m = 950

    for k in steps:
        X_OFF = [x[::k] for x in
                 np.concatenate((X1_0[0:m], X2_0[0:m], X3_0[0:m], X4_0[0:m],
X5_0[0:m], X6_0[0:m], X7_0[0:1950]))]
        X_1 = [x[::k] for x in X1_1[0:m]]
        X_2 = [x[::k] for x in X2_1[0:m]]
        X_3 = [x[::k] for x in X3_1[0:m]]
        X_4 = [x[::k] for x in X4_1[0:m]]
        X_5 = [x[::k] for x in X5_1[0:m]]
        X_6 = [x[::k] for x in X6_1[0:m]]
        X_7 = [x[::k] for x in X7_1[0:1950]]

        X = np.concatenate((X_OFF, X_1, X_2, X_3, X_4, X_5, X_6, X_7))

        y = np.concatenate((np.zeros(len(X_OFF), dtype=int),
```

```
                                np.ones(len(X_1), dtype=int),
                                np.ones(len(X_2), dtype=int) * 2,
                                np.ones(len(X_3), dtype=int) * 3,
                                np.ones(len(X_4), dtype=int) * 4,
                                np.ones(len(X_5), dtype=int) * 5,
                                np.ones(len(X_6), dtype=int) * 6,
                                np.ones(len(X_7), dtype=int) * 7))

        # Разделение данных на обучающий и тестовый наборы
        X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2)
        # Создание и обучение модели k ближайших соседей
        knn = KNearestNeighbors(k=3)
        knn.fit(X_train, y_train)
        knn.save_model(f'models/knn_model_{k}.json')
        # Создание и обучение модели дерева решений
        clf = DecisionTreeClassifier(max_depth=4)
        clf.fit(X_train, y_train)
        # Сохранение модели в файл
        clf.save_model(f'models/tree_model_{k}.json')
        # Создание и обучение модели логистической регрессии
        logreg = LogisticRegression()
        logreg.fit(X_train, y_train)
        # Сохранение модели в файл
        logreg.save_model(f'models/logreg_model{k}.json')
        # Создание и обучение модели линейного дискриминантного анализа
        lda = LinearDiscriminantAnalysis()
        lda.fit(X_train, y_train)
        # Сохранение модели в файл
        lda.save_model(f'models/lda_model{k}.json')
```

Листинг 3 — итоговый код сервера

```
import socket
import numpy as np
import copy
from knn import KNearestNeighbors

knn_model = KNearestNeighbors(load_from_file='models/knn_model_50.json') # 50

def preprocessing(X):
    x = copy.deepcopy(X)
    median_0 = np.median(x[0:1000])
    for i in range(1000, 5000, 1000):
        median = np.median(x[i:i+1000])
        delta = median_0 - median
        if abs(delta) > 1:
            x[i:i+1000] += delta
    return x

resp_dict = {
    0: "Сигнал не обнаружен (Класс 0)",
    1: "Класс 1",
    2: "Класс 2",
    3: "Класс 3",
    4: "Класс 4",
    5: "Класс 5",
    6: "Класс 6",
    7: "Тестовый пульт (Класс 7)"
}

def classify(file_data):
    data = file_data.strip().split(' ')
```

```
    if len(data) != 5000:
        print(len(data))
        return "Некорректная строка"
    signal = preprocessing([float(x) for x in data])

    responses = []
    y_pred = []
    y_pred.append(knn_model.predict(np.array([signal[::50]]))[0])
    print("knn    ->", y_pred[0])
    print("------------------")
    most_common = y_pred[0]
    print("most common", most_common)
    responses.append(resp_dict[most_common])

    return " ".join(responses)

def handle_client(client_socket):
    while True:
        file_data = client_socket.recv(46096).decode('utf-8')
        if not file_data:
            print("Клиент отключился")
            break
        responses = classify(file_data)
        client_socket.sendall(responses.encode('utf-8'))
    client_socket.close()

def start_server():
    server_socket = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
    server_socket.bind(('127.0.0.1', 12345))
    server_socket.listen()
    print("Сервер запущен. Ожидание подключения...")

    while True:
        try:
            client_socket, client_address = server_socket.accept()
            print(f"Подключение установлено с {client_address}")
            handle_client(client_socket)
        except KeyboardInterrupt:
            print("KeyboardInterrupt")
            server_socket.close()
            break

if __name__ == "__main__":
    start_server()
```